we could directly work with weak bisimilarity which is its main semantics, without the need to study the other semantics as in this thesis.

## 2.1 Communicating Sequential Processes

The process algebra CSP (Hoare [1978], Roscoe [1998], Schneider [1999]) was developed by Hoare in Hoare [1978]. CSP is a formal specification language, developed in order to describe concurrent systems by identifying their behaviour through their communications. CSP is a notation for studying processes which interact with each other and their environment. In CSP, we can describe a process by the way it can communicate with its environment. The most fundamental object in CSP is an event. Events can be external events, which can be observed externally and are given by a label, silent internal events, which are not observable from the outside, and termination events, corresponding to the termination of a process. The set of labels forms an alphabet $\Sigma$. A system is formed by one or more processes, where these processes interact with each other through their external and termination events. The overall system in CSP can described by specifying the behaviour of interacting processes. CSP has a variety of semantics which give meaning to the processes.

Similar to CSP, the Calculus of Communicating Systems (CCS) notation Milner [1982], introduced in 1980 by Robin Milner, deals with interacting behaviours of the finite state machine. In CCS processes are defined as agents and the behaviour of processes is defined in term of events they can perform. In CCS a labelled transition system is used to interpret the expressions of the language. Semantic equivalence is defined as bisimilarity, whereas semantic equivalence in CSP is based on extensions of trace semantics. The process algebra CCS, unlike CSP, CCS does not distinguish between external and internal choice operator, whereas CSP has two kind of operators for choices, namely the external choices operator and the internal choices operator. In CSP the internal event never appears in trace semantics, by contrast, internal events in CCS appear in the trace semantics. In CSP there is a process STOP which represents deadlock and a process SKIP which represents successful termination. On the other hand, CCS does not distinguish between deadlock and successful termination. Instead it has a single "terminated" process, nil, which stands for both successful termination and deadlock.

In the following we will first present the syntax of CSP and then introduce different semantics for it.

## 2.2 Framework of CSP

We will first introduce the language of CSP operationally, which is defined in terms of how CSP processes are to be executed. By nature the language of CSP is denotational in nature (Schneider [1999], p. viii). Schneider in Schneider [1999] elucidates the language of CSP as well operationally, in order to obtain a better understanding of the CSP operators. There he introduces the transitions, events, processes, together with inference rules for deriving those transitions.

### 2.2.1 Transitions

The transition systems represent the behaviour of a process. Processes in CSP form a labelled transition system, where the one step transition is written as

$$P \xrightarrow{e} Q \qquad \text{where } P, Q \text{ are processes and } e \text{ is an event.}$$

which means that process $P$ can evolve to process $Q$ by event $e$. The event $e$ can be a label, it can be the silent transition $\tau$, or it can be a termination event $\checkmark$. For example the execution of the process $a \longrightarrow b \longrightarrow STOP$ can be described by the LTS:

$$(\ a \longrightarrow b \longrightarrow STOP) \xrightarrow{a} (b \longrightarrow STOP) \xrightarrow{b} STOP$$

### 2.2.2 Inference rules

The inference rules of CSP are determined by inference rules of the form

$$\frac{\text{Antecedent}_1 \quad \ldots \quad \text{Antecedent}_\text{n}}{\text{Conclusion}} \text{(Sidecondition)}$$

Derivations of a transition are the derivations formed from those inference rules and are inductively defined.

The operational semantics of CSP defines processes as states. Transitions between states are the firing rules for those processes.

### 2.2.3 Events

A system in CSP is given by its components (processes). These components can interact with each other or with the environment by their interfaces. For instance, a cassette recorder might be considered as a process. Its interface will include buttons like *Play*, *Stop*, *Forward*, *Backward* etc., and as well

the door through which the tape can be inserted. Through this interface the process can be interact with the users (environment). This interface is considered as a set of events. The set of events in a CSP process interface are classified as a static specification, and considered as dynamic specifications by describing how it behaves operationally.[1] The events in CSP represent interactions, and are considered to be indivisible and instantaneous. The events in CSP could be atomic. For instance, the interface of the last example, the cassette recorder, has atomic events *Play*, *Stop*, *Forward*, *Backward*, etc. Events in CSP can be as well compound, i.e. it can have some structure. For instance, consider the Automated Teller Machine (ATM) as a process, where the customer can enter the card and return it, by ignoring the other events. This can be written in CSP as having the events $card.in$ and $card.out$. Additionally, event can have some structure by a communication channel which carries some information (i.e. messages). The value $v$ communicated over a channel $c$, can be described as an event $c.v$ in the process interface. For instance, consider the processes $P$ with input channel $in$ which can carry the values 0 and 1. Both values 0 and 1 carried over the channel $c$ may appear in the interface of the process $P$ as separate events $in.0$ and $in.1$. Furthermore an event can be considered as an input and output event. For instance, consider the following CSP expression:

$$c!v \longrightarrow P$$

The above process is willing to output the value $v$ over channel $c$, and then behave like a process $P$. The behaviour is captured by the following firing rule:

$$\frac{}{(c!v \ \longrightarrow P) \xrightarrow{c.v} P}$$

Input phenomena can be explained through the following expression:

$$c?x : T \longrightarrow P(x)$$

Here a processes $P(x)$ is defined for each $x \in T$. The process $c?x : T \longrightarrow P(x)$ is willing to received any value $x$ of the type $T$ over the channel $c$, and then behave like a process $P(x)$. The receiving phenomena is described in CSP by the following firing rule:

$$\frac{}{(c?x : T \ \longrightarrow P(x)) \xrightarrow{c.v} P(v)} \ [v \in T]$$

[1] We note here that in CSP-Agda we will index the possible events by index sets. These index sets will not carry any semantic meaning.

Besides this kind of external events, which are labelled by elements of an alphabet $\Sigma$ , there are internal events labelled by $\tau$, which are internal changes of the state of a process. Furthermore there are termination events $\checkmark$, which will be introduced in Subsect. 2.3.1 below.

In this thesis, we assume an alphabet $\Sigma$, where $\checkmark, \tau \notin \Sigma$. $\mu$ will be an element of $\Sigma^{\checkmark,\tau} := \Sigma \cup \{\checkmark, \tau\}$, and $a, b, c$ will be elements of $\Sigma$.

## 2.3 The Syntax of CSP

In this section we will explain each operator and give an example to illustrate its meaning. In the following table, we list the syntax of CSP process i.e. the primitive CSP processes and the CSP operations. Here Q represents CSP processes:

| | | |
|---|---|---|
| Q ::= | $STOP$ | deadlock process |
| \| | $SKIP$ | terminating process |
| \| | $DIV$ | diverging process |
| \| | $a \longrightarrow Q$ | event prefix process |
| \| | $Q \,\Box\, Q$ | external choice |
| \| | $Q \sqcap Q$ | internal choice |
| \| | $Q \setminus a$ | hiding |
| \| | $Q[R]$ | renaming |
| \| | $Q \;\|[\, A \mid B \,]\|\; Q$ | alphabetised parallel |
| \| | $Q \;\|\|\|\; Q$ | interleaving |
| \| | $Q \,\|[\, A \,]\|\, Q$ | interface parallel |
| \| | $Q \,\triangle\, Q$ | interrupt |
| \| | $Q;\; Q$ | composition |

In the above table, $A$ and $B$ are parameters, which represent a set of events that $Q_{left}$ and $Q_{right}$ are respectively allowed to perform. Furthermore, $R$ is a function for renaming event in the process $Q$.

### 2.3.1 Primitive Processes

CSP has many primitive, i.e. atomic processes. The main primitive process is $STOP$. The process $STOP$ does not have any transition rule since it can not engage in any event and refuses all communication. The process $STOP$ represents the deadlock process, which has no (internal or external) transitions.

CSP has as well the process $SKIP$, which represents successful termination (i.e. reached terminations). This is indicated by performing a special

event called the termination event ✓. As the ✓ is a special event, it can not be a member of the alphabet of the process (i.e., it can not be appear as a part of prefix operator). The CSP process *SKIP* terminates immediately. It has the following rule:

$$\frac{}{SKIP \xrightarrow{\checkmark} STOP}$$

Furthermore we have the diverging *DIV*. This process has as only transition

$$\frac{}{DIV \xrightarrow{\tau} DIV}$$

which means it can perform an infinite sequence of $\tau$-transitions.

### 2.3.2 Sequential Composition

The sequential composition of two processes $P$ and $Q$ is written in CSP as $P\ ;\ Q$. The combination $P\ ;\ Q$ behaves as $P$ until it terminates, in which case it switches to $Q$. For instance, the process

$$COMP_1 \ = \ a \longrightarrow SKIP\ ;\ \ b \ \longrightarrow STOP$$

perform the event $a$ followed by a $b$ event and then behaves like a process *STOP*, i.e. it deadlocks. This process behaves as $a \ \rightarrow \ b \rightarrow STOP$. The following processes

$$COMP_2 \ = \ a \longrightarrow STOP\ ;\ \ b \ \longrightarrow STOP$$

behaves in different way: after performing event $a$ the process behaves like a process *STOP* (i.e. deadlock), which never terminates successfully. Therefore control is never passed over to process $b \ \longrightarrow STOP$, so $COMP_2$ behaves as $a \longrightarrow STOP$.

The transition rules for the sequential composition operator are as follows:

$$\frac{P \xrightarrow{\checkmark} \bar{P}}{P;\ Q \xrightarrow{\tau} Q} \qquad \frac{P \xrightarrow{\mu} \bar{P}}{P;\ Q \xrightarrow{\mu} \bar{P};\ Q}\ [\mu \neq \checkmark]$$

The sequential composition operates initially like the first process. When it terminated, the successful termination event becomes internal, and after that internal event the process behaves like $Q$. The process composed by sequential composition has only a termination event once it has reached a termination event of the 2nd process.

### 2.3.3 Event Prefix

The prefix operator constructs from an existing process $P$ a process $a \longrightarrow P$, pronounced in CSP 'a then $P$'. It has one external event $a \in \Sigma$ after which it behaves like $P$. Its behaviour is captured by the following firing rule:

$$\frac{}{(a \longrightarrow P) \xrightarrow{a} P}$$

This is an example of an axiom, i.e. a rule with no side conditions or antecedents.

For instance, consider then process *PreF* which can perform only an event $a$ and then behave like the process *STOP* having no transitions. It is given as follows:

$$PreF \;=\; a \longrightarrow STOP$$

### 2.3.4 Recursion

CSP uses recursive definitions in order to describe the execution of processes, which operate in an infinite manner. Recursive processes are introduced in CSP in an equational style $N = P$. Here $N$ is a new name for the process to be defined, and $P$ is a CSP expression which can make use of $P$. This recursive definition of processes will work only if the righthand side of the equation begins with at least one event prefixed to all recursive occurrences of the process name. A example where this condition is not fulfilled would be the recursive equation $N = N$, which doesn't define a valid process. A process which begins with a prefix is called a guarded process, and if the right hand side of a recursive definition is a guarded processes the recursive definition has a unique solution.

For example $P = a \longrightarrow b \longrightarrow P$ is a process that repeats infinitely often an $a$ event followed by an $b$. The firing rule for unwinding a process $N$ recursively bound to a definition of process $P$ is given as follows:

$$\frac{P \xrightarrow{\mu} \bar{P}}{N \xrightarrow{\mu} \bar{P}} [N = P]$$

The rule above states that the transitions from $P$ are the same as those originating from $N$. We can write the process $P$ in the form of $F(N)$ in order to make the dependency on $N$ explicit. Then we have $F(Y) = P[Y/N]$ where the latter is the substitution of $N$ by $Y$. As an example we have

$$a \longrightarrow b \longrightarrow P\ [a \longrightarrow STOP/P] = a \longrightarrow b \longrightarrow a \longrightarrow STOP$$

### 2.3.5 Internal Choice

CSP offers two kinds of operators for choices between two processes: external choice and internal choice. In external choice the operator allows the environment to choose an external event of $P$ or $Q$, and then continue as process $P$ or $Q$, respectively. In contrast, internal choice leaves the choice between processes to the process. The internal choice between processes $P$ and $Q$ is written as $P \sqcap Q$. For this processes the process chooses non-deterministically to continue as $P$ or as $Q$. Consider for instance the process

$$IntCH \;=\; a \longrightarrow SKIP \;\sqcap\; b \;\longrightarrow STOP$$

In this case the process switches internally to either $a \longrightarrow SKIP$ or $b \longrightarrow STOP$. In the first case it has only one external event, namely an $a$-event, and then behaves like $SKIP$ (i.e. successfully termination), or it has only one $b$ event then behaves like $STOP$ (i.e. deadlock). The inference rule for the internal choice operator as follows:

$$\frac{}{P \;\sqcap\; Q \xrightarrow{\tau} \bar{P}} \qquad\qquad \frac{}{P \;\sqcap\; Q \xrightarrow{\tau} \bar{Q}}$$

As for external choice, CSP has an indexed internal operator, which take an arbitrary number of processes. For instance the process

$$\sqcap_{x \in \{a\ b\ c\}} x \longrightarrow STOP$$

can internally choose $x = a$, or $x = b$ or $x = c$, and then have an external event labelled by $x$ followed by the $STOP$ process.

The inference rule for the indexed internal choice operator is as follows:

$$\frac{}{\sqcap_{i \in J} P_i \xrightarrow{\tau} P_j}\ [(j \in J)]$$

### 2.3.6 External Choice

External choice between processes $P$ and $Q$ is written in CSP as $P \;\Box\; Q$. External choice allows the environment to choose an external event of $P$ or $Q$, and then continue as process $P$ or $Q$, respectively. For instance, consider the process $ExtCH$

$$ExtCH \;=\; a \longrightarrow SKIP \;\;\Box\;\; b \;\longrightarrow STOP$$

The $ExtCH$ process can firstly perform one of two event $a$ or $b$. If $a$ is chosen it continues as $SKIP$, and if $b$ is chosen it continues as $STOP$. The firing rule for the external choice are as follows:

$$\frac{P \xrightarrow{a} \bar{P}}{\begin{array}{c} P \,\Box\, Q \xrightarrow{a} \bar{P} \\ Q \,\Box\, P \xrightarrow{a} \bar{P} \end{array}} \qquad \frac{P \xrightarrow{\tau} \bar{P}}{\begin{array}{c} P \,\Box\, Q \xrightarrow{\tau} \bar{P} \,\Box\, Q \\ Q \,\Box\, P \xrightarrow{\tau} Q \,\Box\, \bar{P} \end{array}}$$

There exist as well a generalisation of the binary external choice operator, which creates the external choice over a finite number of processes indexed over an index set. The firing rules for the indexed external choice operator are as follows:

$$\frac{P_j \xrightarrow{a} \bar{P}}{\Box_{i \in I} \; P_i \xrightarrow{a} \bar{P}} \, [j \in I] \qquad \frac{P_j \xrightarrow{\tau} \bar{P}_j}{\Box_{i \in I} \; P_i \xrightarrow{\tau} \Box_{i \in I} \; \bar{P}_i} \, [j \in I]$$

### 2.3.7 Parallel and Interleaving

CSP has several parallel operator which enforce a number of processes to work together and interact through synchronous events. When the participants engage in the synchronous events, synchronisation occurs simultaneously, which is like a a handshake, and therefore called *handshake synchronisation*. In the following we will explain the most important parallel operators together with examples.

#### Alphabetised Parallel

The alphabetised parallel operator of CSP depends on alphabets $A, B$ for the two processes $P, Q$ operating in parallel. When put in parallel, process $P$ can only perform events in $A$ and process $Q$ can only perform events in $B$. Furthermore, any event in $A \cap B$ needs to be synchronised between $P$ and $Q$. Silent transitions can occur individually in $P$ and in $Q$, whereas the termination event needs to be synchronised between $P$ and $Q$. The transition rules are as follows:

$$\frac{P \xrightarrow{a} \bar{P} \qquad Q \xrightarrow{a} \bar{Q}}{P \;|[\, A \,|\, B \,]|\; Q \xrightarrow{a} \bar{P} \;|[\, A \,|\, B \,]|\; \bar{Q}} \, [\; a \in (A \cup \{\checkmark\}) \cap (B \cup \{\checkmark\}) \;]$$

$$\frac{P \xrightarrow{\mu} \bar{P}}{\begin{array}{c} P \;|[\, A \,|\, B \,]|\; Q \xrightarrow{\mu} \bar{P} \;|[\, A \,|\, B \,]|\; Q \\ Q \;|[\, B \,|\, A \,]|\; P \xrightarrow{\mu} Q \;|[\, B \,|\, A \,]|\; \bar{P} \end{array}} \, [\; \mu \in (A \cup \tau) \backslash B]$$

The first rule states that events in the interaction between $A$ and $B$ and termination events need to be executed by both processes simultaneously.

The second rule allows each process to perform events, which are not in the alphabet of the other process and silent transitions individually.

As an example consider

$$(a \longrightarrow P) \ |[\,\{a\}\,|\,\{a,b\}\,]|\ (a \longrightarrow Q \ \ \Box\ b \longrightarrow Z)$$

This process can perform the event $a$ and behave as a process $P \ |[\,\{a\}\,|\,\{a,b\}\,]|\ Q$, or perform the event $b$ and behave as $(a \longrightarrow P) \ |[\,\{a\}\,|\,\{a,b\}\,]|\ Z$.

Consider consider the following process:

$$(a \longrightarrow P) \ |[\,\{a,b\}\,|\,\{a,b\}\,]|\ (a \longrightarrow Q \ \ \Box\ b \longrightarrow Z)$$

It cannot perform event $b$, since the left process cannot perform it. It can only perform event $a$ and then behave as a $P \ |[\,\{a,b\}\,|\,\{a,b\}\,]|\ Q$.

### Interleaving

Independent concurrent behaviour is represented in CSP by the interleaving operator. The interleaving of $P$ and $Q$ is written in CSP as $P \ |||\ Q$ and is pronounced as "$P$ interleave $Q$". The combination $P \ |||\ Q$ executes processes $P$ and $Q$ independently. The only way the processes can engage with each other is when they terminate. The transition rules are as follows:

$$\frac{P \xrightarrow{\checkmark} \bar{P} \qquad Q \xrightarrow{\checkmark} \bar{Q}}{P \ |||\ Q \xrightarrow{\checkmark} \bar{P} \ |||\ \bar{Q}}$$

$$\frac{P \xrightarrow{\mu} \bar{P}}{P \ |||\ Q \xrightarrow{\mu} \bar{P} \ |||\ Q}\ [\,\mu \neq \checkmark]$$

$$Q \ |||\ P \xrightarrow{\mu} Q \ |||\ \bar{P}$$

If $P$ and $Q$ have disjoint alphabets $A$, $B$ , the alphabetised parallel behaves as interleaving: If $P$ and $Q$ have events in $A$, $B$ respectively, and $A \cap B = \emptyset$ then $P \ \ |[\,A\,|\,B\,]|\ Q$ and $P \ |||\ Q$ behave in the same way.

For instance

$$(a \longrightarrow b \longrightarrow P) \ |[\,\{a,b\}\,|\,\{c,d\}\,]|\ (c \longrightarrow d \longrightarrow Q\ )$$
behaves as
$$(a \longrightarrow b \longrightarrow P) \ |||\ (c \longrightarrow d \longrightarrow Q\ )$$

**Interface Parallel**

The interface parallel operator defines the parallel operation of two processes, where a subset of the interface is synchronised. So the synchronised event can only occur when all processes are able to perform it. The operational semantics for the interface parallel operator is given by the following rules:

$$\frac{P \xrightarrow{a} \bar{P} \qquad Q \xrightarrow{a} \bar{Q}}{P \,|[\, A \,]|\, Q \xrightarrow{a} \bar{P} \,|[\, A \,]|\, \bar{Q}} \, [a \in A \cup \checkmark]$$

$$\frac{P \xrightarrow{\mu} \bar{P}}{P \,|[\, A \,]|\, Q \xrightarrow{\mu} \bar{P} \,|[\, A \,]|\, Q} \, [\, \mu \notin A \cup \checkmark]$$

$$Q \,|[\, A \,]|\, P \xrightarrow{\mu} Q \,|[\, A \,]|\, \bar{P}$$

Consider for instance $P = a \longrightarrow b \longrightarrow P$, $Q = a \longrightarrow c \longrightarrow Q$. The only transition of $P \;|[\,\{a\}\,]|\; Q$ is

$$P \;|[\,\{a\}\,]|\; Q \xrightarrow{a} (b \longrightarrow P) \,|[\,\{a\}\,]|\, (c \longrightarrow Q)$$

After this transition it can continue in two different ways:

$$(b \longrightarrow P) \,|[\,\{a\}\,]|\, (c \longrightarrow Q) \xrightarrow{b} P \,|[\,\{a\}\,]|\, (c \longrightarrow Q) \xrightarrow{c} P \,|[\,\{a\}\,]|\, Q$$

and

$$(b \longrightarrow P) \,|[\,\{a\}\,]|\, (c \longrightarrow Q) \xrightarrow{c} (b \longrightarrow P) \,|[\,\{a\}\,]|\, Q \xrightarrow{b} P \,|[\,\{a\}\,]|\, Q$$

### 2.3.8 Hiding

The hiding operator, hides a set of events in a process, replacing them by internal $\tau$-transitions. The main usage is when processes communicate with each other, and one wants to hide these internal communications. The notation in CSP for hiding is $P \backslash A$ pronounced "$P$ hides $A$", which hides the events in $A$ from process $P$. Since the events are no longer visible, no other process can engage with them. The behaviour of the hiding operator is determined by the following firing rules:

$$\frac{P \xrightarrow{a} \bar{P}}{P \setminus A \xrightarrow{\tau} \bar{P} \setminus A} \, [\, a \in A] \qquad \frac{P \xrightarrow{\mu} \bar{P}}{P \setminus A \xrightarrow{\mu} \bar{P} \setminus A} \, [\, \mu \notin A]$$

For example, the protocol stop-and-wait (based on examples taken from Schneider [1999] Section 3.1) implements a one-place buffer as a process consists of two parts *Sender* and *Receiver*. When its combined through the parallel operator a one-place buffer is formed. A messages input to the *Sender* is passed over the channel *mid* to the *Receiver*, then the *Sender* waits for acknowledgement. The processes *Sender* and *Receiver* are specified as follows:

$$\begin{aligned} Sender &= \quad in?x \longrightarrow mid!x \longrightarrow ack \longrightarrow Sender \\ Receiver &= mid?y \longrightarrow out!y \longrightarrow ack \longrightarrow Receiver \end{aligned}$$

The *Sender* reads a value of type T over channel *in* and binds it to the variable $x$. Afterwards this value is sent over channel *mid* to *Receiver*. When the message has been output, *Receiver* sends an acknowledgement. The combined behaviour is obtained by combining processes *Sender* and *Receiver* using the parallel operator, and then hiding the internal communications, as follows:

$$(Sender \parallel Receiver) \setminus (mid \cup ack)$$

In the above definition *Sender* and *Receiver* are combined using the parallel operator and all communication along channels *mid* and via event *ack* are hidden. The resulting process can be considered as a a black box regarding events *mid* and *ack*. Externally it can receive value over a channel *in* and output it on channel *out*.

### 2.3.9 Renaming

The renaming operator allows to rename the events in a process. This operator is useful to obtain a new process by reusing previous description of processes without the need for rewriting the processes in full. In the language of CSP a total bijective function ($f : \Sigma \cup \checkmark \longrightarrow \Sigma \cup \checkmark$) on events is used to described the changing of the events, with the additional condition that $f(\checkmark) = \checkmark$. The renaming function can map external events to external events, but not to internal events. Termination events are not changed by the renaming function. The operational semantics of renaming operator is captured by the following transition rules:

$$\frac{P \xrightarrow{a} \bar{P}}{f(P) \xrightarrow{f(a)} f(\bar{P})} \qquad \frac{P \xrightarrow{\tau} \bar{P}}{f(P) \xrightarrow{\tau} f(\bar{P})}$$

### 2.3.10 Interrupt

The interrupt operator $P \triangle Q$ offers the mechanism of passing the control from one process to another. The difference between $P \triangle Q$ and sequential composition is the control in $P \triangle Q$ can pass from $P$ to $Q$ by interrupt at any time (i.e. by $Q$ stealing control). The interrupt operator in the process $P \triangle Q$ allows the control to remain with P while the events are carried out by P. The moment an external event is performed by Q, the control passes to Q. The inference rules for the interrupt operator are as follows:

$$\frac{P \xrightarrow{\mu} \bar{P}}{P \triangle Q \xrightarrow{\mu} \bar{P} \triangle Q}\,[\mu \neq \checkmark] \qquad \frac{P \xrightarrow{\checkmark} \bar{P}}{P \triangle Q \xrightarrow{\checkmark} \bar{P}}$$

$$\frac{Q \xrightarrow{\tau} \bar{Q}}{P \triangle Q \xrightarrow{\tau} P \triangle \bar{Q}} \qquad \frac{Q \xrightarrow{a} \bar{Q}}{P \triangle Q \xrightarrow{a} \bar{Q}}$$

## 2.4 The Semantics of CSP

The main goal of a semantics for CSP is to determine whether two processes P and Q are equal, and whether process P refines process Q. These two conditions are in fact equivalent, both can be formulated in terms of the other, and therefore it is only necessary to define one of them in a semantics. CSP has different kinds of semantics: operational semantics, denotational semantics, and axiomatic semantics. Each semantics gives a meaning to the expressions.

Operational semantics is used to create a transition system for a CSP process. The axiomatic semantics allows to derive facts from the derivation rules. It offers a set of algebraic laws to transform processes into other equal processes, and can be used to prove that two different processes are equal. Denotational semantics determines a semantic domain and for each process and element of this domain as its semantics. CSP has according to Schneider [1999] three main denotational semantics: (1) trace semantics(*traces*), (2) stable failures semantic (*failures*), and (3) failures, divergences, and infinite traces semantics (*failures*, *divergences*, *infinites*). Apart of this there exist three main kind of bisimulation semantics: strong bisimulation, weak bisimulation, and divergence-respecting weak bisimulation.

### 2.4.1 Trace Semantics

In CSP traces of a process are the sequences of actions, i.e. the labels of external choices, a process can perform. We simply record the actions that